

MODUL 3

Blade Template Laravel



Disusun Oleh:

Akmallutfi Afza Gusty

NIM: 707012500064

Kelas: SIKC 49 - 04

Mata Kuliah:

Pengembangan Aplikasi Berbasis Web

Dosen Pengampu:

Diash Firdaus

Program Studi Sistem Informasi Kota Cerdas

1. Tujuan Praktikum

Melalui praktikum Modul 3 ini, saya ingin mencapai beberapa kemampuan berikut. Pertama, memahami konsep Blade Template sebagai template engine bawaan Laravel beserta cara kerja Laravel mengompilasi sebuah view menjadi kode PHP yang disimpan di cache. Kedua, mampu menuliskan sintaks dasar Blade, terutama `{{ }}` untuk menampilkan data. Ketiga, mampu memanfaatkan directive Blade untuk percabangan (`@if`, `@elseif`, `@else`) dan perulangan (`@foreach`, `@forelse`) agar tampilan dapat menyesuaikan data yang dikirim oleh controller.

Keempat, mampu membangun tampilan web yang seragam dan mudah dirawat dengan fitur layouting `@extends`, `@section`, dan `@yield`. Kelima, mampu membuat dan memakai kembali potongan tampilan melalui komponen Blade (`<x-...>`) dan partial (`@include`). Terakhir, saya ingin menerapkan seluruh konsep tersebut pada studi kasus aplikasi pelaporan banjir “LaporBanjir” milik BPBD Kabupaten Bandung.

2. Alat dan Bahan

Praktikum ini dikerjakan dengan perangkat dan perkakas berikut.

Perangkat Keras:

- Laptop dengan prosesor Intel Core i5, RAM 8 GB, dan penyimpanan SSD 512 GB.
- Sistem operasi Windows 11 (64-bit).

Perangkat Lunak:

Perangkat Lunak	Versi / Keterangan
Laragon	Server lokal Apache & MySQL (full)
PHP	8.5.10 (NTS, Win32-vs17-x64)
Composer	2.10.3 (dependency manager PHP)
Laravel Framework	13.17
Node.js	24.14.0 (untuk Vite)
Visual Studio Code	Teks editor / IDE
Google Chrome	Browser pengujian
Git	2.53.0, kontrol versi

3. Langkah-Langkah Praktikum

Pengerjaan tugas praktikum dilakukan dengan pola MVC (Model–View–Controller). Karena masih berupa prototipe, data tidak diambil dari database, melainkan ditulis dalam bentuk array di dalam controller. Aplikasi yang dibangun adalah sistem pelaporan banjir “LaporBanjir” yang terdiri atas tiga halaman, yaitu form pelaporan, halaman konfirmasi, dan halaman daftar laporan.

3.1 Persiapan Proyek dan Struktur Folder

Aplikasi dibuat di dalam proyek Laravel yang telah disiapkan. Struktur view yang dibangun mengikuti pola pemisahan layout, halaman, komponen, dan partial seperti diperlihatkan pada potongan struktur berikut.

```
resources/views/  
├─ layouts/  
|   └─ app.blade.php          // layout utama LaporBanjir  
├─ components/  
|   └─ alert.blade.php        // komponen pesan <x-alert>  
├─ partials/  
|   └─ laporan-card.blade.php // partial kartu laporan banjir  
└─ laporan/  
    ├── form.blade.php        // halaman form pelaporan  
    ├── konfirmasi.blade.php  // halaman konfirmasi  
    └─ index.blade.php        // halaman daftar laporan
```

Langkah awal adalah membuat controller dengan perintah artisan, kemudian membuat file-file view secara manual di dalam folder `resources/views`.

```
php artisan make:controller laporanBanjirController
```

3.2 Konfigurasi Route

Route menghubungkan alamat URL dengan method pada controller. Setiap halaman diberi named route agar dapat dipanggil dari Blade dengan helper `route()` sehingga tautan navigasi tidak ditulis manual. Isi `routes/web.php`:

```
<?php  
  
use Illuminate\Support\Facades\Route;  
use App\Http\Controllers\LaporanBanjirController;  
  
Route::get('/', function () {  
    return view('welcome');  
});
```

```
});

Route::get('/laporan', [laporanBanjirController::class,
'index'])->name('laporan.index');
Route::get('/laporan/create',
[laporanBanjirController::class, 'create'])-
>name('laporan.create');
Route::post('/laporan',
[laporanBanjirController::class, 'store'])-
>name('laporan.store');
```

Alamat utama / menampilkan halaman welcome bawaan Laravel. Halaman daftar laporan diakses dengan method GET pada /laporan, sedangkan halaman form diakses dengan method GET pada /laporan/create. Form kemudian dikirim melalui method POST ke laporan.store. Dengan cara ini seluruh halaman dapat diakses melalui route masing-masing dan dipanggil memakai named route di dalam view.

3.3 Pembuatan Controller

Controller berperan menyiapkan data dan mengolah request. Pada laporanBanjirController terdapat method dataLaporan() yang menyimpan contoh data laporan dalam bentuk array, method create() untuk menampilkan form, store() untuk memproses data POST, serta index() untuk daftar laporan. Isi controller:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class laporanBanjirController extends Controller
{
    public function dataLaporan()
    {
        return [
            [
                'nama' => 'Budi Santoso',
                'lokasi' => 'Desa Cangkuang,
Dayeuhkolot',
                'tinggi' => 25,
                'waktu' => '07:45 WIB',
```

```

        ],
        [
            'nama' => 'Siti Aminah',
            'lokasi' => 'Jl. Raya Soreang KM 10',
            'tinggi' => 45,
            'waktu' => '08:20 WIB',
        ],
        [
            'nama' => 'Rudi Hartono',
            'lokasi' => 'Kecamatan Baleendah',
            'tinggi' => 60,
            'waktu' => '09:05 WIB',
        ],
        [
            'nama' => 'Dewi Lestari',
            'lokasi' => 'Desa Bojongsoang',
            'tinggi' => 85,
            'waktu' => '09:40 WIB',
        ],
        [
            'nama' => 'Agus Setiawan',
            'lokasi' => 'Kelurahan Andir,
Baleendah',
            'tinggi' => 120,
            'waktu' => '10:15 WIB',
        ],
    ],
];

public function create()
{
    return view('laporan.form');
}

public function store(Request $request)
{
    $laporan = [
        'nama' => $request->nama,
    ];
}

```

```

        'lokasi' => $request->lokasi,
        'tinggi' => $request->tinggi,
    ];

    return view('laporan.konfirmasi',
compact('laporan'));
}

public function index()
{
    $laporan = $this->dataLaporan();

    return view('laporan.index',
compact('laporan'));
}
}

```

Fungsi `compact()` memastikan data dikirim dari controller ke view, sehingga tidak ada logika PHP di dalam file tampilan. Method `store()` membaca input form melalui objek `$request` lalu mengirimkannya kembali ke view konfirmasi.

3.4 Layout Utama (Layouting dengan `@extends`, `@section`, `@yield`)

Layout utama `layouts/app.blade.php` menyimpan bagian yang sama di setiap halaman, yaitu header berisi nama sistem, menu navigasi memakai named route, footer, dan CSS. Bagian yang berubah ditandai dengan `@yield`. Isi layout:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>@yield('title') | BPBD Bandung</title>

    <style>
        body{font-family: Arial, Helvetica, sans-serif;
margin: 20px;}
        header, footer{background: #f5f5f5; padding:
10px; text-align: center;}
    </style>

```

```

        nav a {margin: 0 10px; text-decoration: none;}
        .card {border: 1px solid #ddd; padding: 15px;
margin: 10px 0; border-radius: 6px;}
        .btn {background: #007bff; color: white;
padding: 8px 14px; border: none; border-radius: 4px;
text-decoration: none; cursor: pointer;}
        .btn-secondary {background: #6c757d;}
        label {display: block; margin: 8px 0 4px;}
        input {padding: 6px; width: 300px;}
        .data-list dt {font-weight: bold; margin-top:
8px;}

        .data-list dd {margin: 0 0 4px;}
        .tandaWaspada {color: green;}
        .tandaSiaga {color: orange;}
        .tandaAwat {color: red;}
    </style>
</head>
<body>
    <header>
        <h1>Lapor Banjir</h1>
        <nav>
            <a href="{{ route('laporan.create')
}}">Form Pelaporan</a>
            <a href="{{ route('laporan.index')
}}">Daftar Laporan</a>
        </nav>
    </header>

    <main>
        @yield('content')
    </main>

    <footer>
        <p>&copy; 2026 praktikum laravel TA -
Akma'llutfi Afza Gusty</p>
    </footer>
</body>
</html>

```

Setiap halaman kemudian mewarisi layout ini menggunakan `@extends('layouts.app')`, mengisi judul dengan `@section('title', ...)`, dan mengisi konten dengan `@section('content') ... @endsection`. Bagian `@yield('title')` dan `@yield('content')` inilah yang akan diisi oleh masing-masing view turunan.

3.5 Halaman Form Pelaporan

Halaman form `laporan/form.blade.php` berisi input nama pelapor, lokasi kejadian, dan tinggi genangan air. Form dikirim dengan method POST ke named route `laporan.store` dan menyertakan token `@csrf`.

3.6 Komponen Blade Alert

```
@extends('layouts.app')

@section('title', 'Form Pelaporan')

@section('content')

<h2>Form Pelaporan Banjir</h2>

<div>
    <p>Silahkan isi form Pelaporan banjir</p>

    <form action="{{ route('laporan.store') }}"
method="POST">
        @csrf

        <label for="nama">Nama Pelapor</label>
        <input type="text" id="nama" name="nama"
required>

        <label for="lokasi">Lokasi Kejadian</label>
        <input type="text" id="lokasi" name="lokasi"
required>

        <label for="tinggi">Tinggi Genangan Air
(cm)</label>
        <input type="number" id="tinggi" name="tinggi"
required>
```

```

        <button type="submit" class="btn">Kirim
Laporan</button>
    </form>
</div>

@endsection

```

Komponen `components/alert.blade.php` dibuat sekali dan dapat dipakai berulang dengan atribut berbeda. Warna border ditentukan memakai operator ternary berdasarkan nilai `$type`.

```

<div style="padding:10px; border:1px solid {{ $type ===
'error' ? 'red' : 'green' }}; margin:10px 0;">
    <strong>{{ ucfirst($type) }}</strong> {{ $message
}}
</div>

```

Penjelasan: `{{ $type === 'error' ? 'red' : 'green' }}` adalah operator ternary untuk menentukan warna border, `{{ ucfirst($type) }}` mengubah huruf pertama menjadi kapital, dan `{{ $message }}` menampilkan pesan yang dikirim saat komponen dipanggil. Komponen ini dipanggil pada halaman konfirmasi dengan `<x-alert type="success" message="..." />`.

3.7 Halaman Konfirmasi

Halaman konfirmasi menampilkan pesan berhasil memakai komponen `<x-alert>` dan mengulang kembali data yang telah dikirim. Status genangan ditentukan dengan directive `@if/@elseif/@else`: kurang dari 30 cm “Waspada”, 30–70 cm “Siaga”, dan lebih dari 70 cm “Awat”. Isi laporan/konfirmasi.blade.php:

```

@extends('layouts.app')

@section('title', 'Konfirmasi Laporan')

@section('content')
<h2>Konfirmasi Laporan</h2>

<x-alert type="success" message="Laporan berhasil
dikirim." />

```

```

<div>
  <h3>Detail laporan yang di kirim</h3>

  <dl class="data-list">
    <dt>Nama Pelapor</dt>
    <dd>{{ $laporan['nama'] }}</dd>

    <dt>Lokasi Kejadian</dt>
    <dd>{{ $laporan['lokasi'] }}</dd>

    <dt>Tinggi Genangan Air</dt>
    <dd>{{ $laporan['tinggi'] }} cm</dd>

    <dt>Status Genangan</dt>
    <dd>
      @if($laporan['tinggi'] < 30)
        <span
class="tandaWaspada">Waspada</span>
      @elseif($laporan['tinggi'] <= 70)
        <span class="tandaSiaga">Siaga</span>
      @else
        <span class="tandaAwes">Awes</span>
      @endif
    </dd>
  </dl>

  <a href="{{ route('laporan.create') }}" class="btn
btn-secondary">Kembali ke Form</a>
  <a href="{{ route('laporan.index') }}"
class="btn">Lihat Daftar Laporan</a>
</div>

@endsection

```

3.8 Partial dan Halaman Daftar Laporan

Satu kartu laporan dipisahkan ke dalam partial `partials/laporan-card.blade.php`. Partial ini hanya berfokus menampilkan satu laporan, dan variabel `$laporan` yang digunakan di dalamnya dikirim melalui `@include`.

```
<div class="card">
    <h3>{{ $laporan['nama'] }}</h3>
    <p><strong>Lokasi:</strong> {{ $laporan['lokasi']
}}</p>
    <p><strong>Tinggi Genangan:</strong> {{
$laporan['tinggi'] }} cm</p>
    <p><strong>Waktu:</strong> {{ $laporan['waktu']
}}</p>

    @if($laporan['tinggi'] < 30)
        <p>Status: <span
class="tandaWaspada">Waspada</span></p>
    @elseif($laporan['tinggi'] <= 70)
        <p>Status: <span
class="tandaSiaga">Siaga</span></p>
    @else
        <p>Status: <span
class="tandaAwat">Awat</span></p>
    @endif
</div>
```

Halaman daftar laporan `laporan/index.blade.php` menampilkan seluruh laporan dengan directive perulangan `@forelse` dan memanggil partial di atas.

```
@extends('layouts.app')

@section('title', 'Daftar Laporan')

@section('content')
    <h2>Daftar Laporan Banjir</h2>

    @forelse($laporan as $item)
        @include('partials.laporan-card', ['laporan' =>
$item])
    @empty
        <p>Belum ada laporan masuk.</p>
    @endforelse
</section>
```

```
@endforelse
@endsection
```

Penjelasan: `@forelse($laporan as $item)` melakukan perulangan terhadap data laporan, `@include('partials.laporan-card', ['laporan' => $item])` memanggil partial dan mengoper variabel `$laporan` ke dalamnya, sedangkan blok `@empty` menampilkan pesan apabila data kosong.

3.9 Menjalankan dan Menguji Aplikasi

Aplikasi dijalankan dengan perintah `artisan serve`, kemudian setiap halaman diuji melalui browser pada alamat `127.0.0.1:8000`. Pengujian meliputi pengisian form dan pengiriman data untuk memastikan halaman konfirmasi menampilkan kembali data dengan benar.

```
php artisan serve
```

```
# kemudian buka:
```

```
# http://127.0.0.1:8000/laporan/create    (form pelaporan)
```

```
# http://127.0.0.1:8000/laporan          (daftar laporan)
```

Catatan lingkup pekerjaan: pada snapshot proyek yang ada saat ini, bagian yang telah diimplementasikan adalah aplikasi `LaporBanjir` (form, konfirmasi, dan daftar laporan) beserta layout, komponen, dan partial-nya. Selain itu masih tersedia route `/` yang menampilkan halaman `welcome` bawaan `Laravel`.

4. Hasil dan Dokumentasi Program

Berikut adalah hasil aplikasi yang telah dijalankan dan diuji melalui browser. Setiap gambar diberi nomor, keterangan, dan penjelasan singkat.

Lapor Banjir

[Form Pelaporan](#) [Daftar Laporan](#)

Form Pelaporan Banjir

Silahkan isi form Pelaporan banjir

Nama Pelapor

Lokasi Kejadian

Tinggi Genangan Air (cm)

[Kirim Laporan](#)

© 2026 praktikum laravel TA - Akmallutfi Afza Gusty

Gambar 4.1 Halaman Form Pelaporan LaporBanjir

Gambar 4.1 menampilkan halaman form pelaporan yang mewarisi `layouts/app.blade.php`. Tampak header berisi nama sistem, menu navigasi (Form Pelaporan dan Daftar Laporan), tiga input (nama pelapor, lokasi kejadian, dan tinggi genangan air), tombol kirim, serta footer. Form mengirim data melalui method POST ke named route `laporan.store`.

Lapor Banjir

[Form Pelaporan](#) [Daftar Laporan](#)

Konfirmasi Laporan

Success Laporan berhasil dikirim.

Detail laporan yang di kirim

Nama Pelapor
Akmal

Lokasi Kejadian
buah batu

Tinggi Genangan Air
12 cm

Status Genangan
Waspada

[Kembali ke Form](#) [Lihat Daftar Laporan](#)

© 2026 praktikum laravel TA - Akmallutfi Afza Gusty

Gambar 4.2 Halaman Konfirmasi dan Komponen Alert

Gambar 4.2 memperlihatkan halaman konfirmasi setelah form dikirim. Pesan keberhasilan ditampilkan menggunakan komponen Blade `<x-alert type="success" ... />`. Data yang dikirim (nama, lokasi, dan tinggi genangan) ditampilkan kembali, dan status genangan dihitung dengan `@if/@elseif/@else` sehingga muncul label **Awas** ketika tinggi melebihi 70 cm.

Lapor Banjir

[Form Pelaporan](#) [Daftar Laporan](#)

Daftar Laporan Banjir

Budi Santoso

Lokasi: Desa Cangkuang, Dayeuhkolot

Tinggi Genangan: 25 cm

Waktu: 07:45 WIB

Status: Waspada

Siti Aminah

Lokasi: Jl. Raya Soreang KM 10

Tinggi Genangan: 45 cm

Waktu: 08:20 WIB

Status: Siaga

Rudi Hartono

Lokasi: Kecamatan Baleendah

Tinggi Genangan: 60 cm

Waktu: 09:05 WIB

Status: Siaga

Dewi Lestari

Lokasi: Desa Bojongsoang

Tinggi Genangan: 85 cm

Waktu: 09:40 WIB

Status: Awas

Agus Setiawan

Lokasi: Kelurahan Andir, Baleendah

Tinggi Genangan: 120 cm

Waktu: 10:15 WIB

Status: Awas

© 2026 praktikum laravel TA - Akmalutifi Afza Gusty

Gambar 4.3 Halaman Daftar Laporan dengan Status Genangan

Gambar 4.3 menampilkan daftar laporan hasil perulangan `@forelse` yang memanggil partial `partials.laporan-card`. Terlihat ketiga kategori status: Waspada (25 cm, hijau), Siaga (45 dan 60 cm, oranye), serta Awas (85 dan 120 cm, merah). Hal ini membuktikan directive percabangan dan perulangan bekerja sesuai ketentuan modul.

5. Analisis dan Pembahasan

Pembagian layout, komponen, dan partial. Layout `layouts/app.blade.php` dibuat untuk menampung bagian yang sifatnya statis dan berulang di setiap halaman, yaitu header, menu navigasi, footer, dan CSS. Dengan cara ini, ketika warna atau menu ingin diubah, saya cukup mengubah satu file dan seluruh halaman yang mewarisinya ikut berubah. Komponen `<x-alert>` dipakai untuk elemen antarmuka kecil yang muncul di banyak tempat (pesan sukses pada halaman konfirmasi) karena komponen menerima atribut sehingga lebih fleksibel. Sementara itu, partial `partials/laporan-card.blade.php` dipakai untuk memecah bagian tampilan yang besar (kartu laporan) agar view utama lebih ringkas dan mudah dibaca. Pembagian ini sesuai panduan modul: gunakan komponen untuk elemen UI kecil yang menerima atribut, dan partial untuk memecah halaman yang panjang.

Perbedaan sebelum dan sesudah memakai Blade. Sebelumnya, tampilan ditulis sebagai file HTML tersendiri yang menuliskan ulang bagian `<head>`, CSS, dan struktur halaman di setiap

file, serta mencampur logika PHP langsung di dalam tampilan. Setelah memakai Blade, setiap halaman cukup menulis `@extends` dan `@section`, data ditampilkan dengan `{{ }}` yang otomatis melakukan escape sehingga lebih aman, dan logika percabangan serta perulangan ditulis dengan directive yang lebih ringkas. Kode pun menjadi lebih konsisten dan mudah dirawat.

Kendala dan cara penyelesaiannya. Selama praktikum ada beberapa kendala yang saya temukan. Pertama, saat memisahkan kartu ke dalam partial, data tidak muncul karena nama variabel di dalam partial harus sama dengan yang dioper oleh `@include`; masalah ini diselesaikan dengan konsisten memakai key 'laporan' sehingga di dalam partial variabelnya adalah `$laporan`. Kedua, dua halaman sempat memakai alamat yang sama pada path `/laporan` (form dan daftar); hal ini diatasi dengan memindahkan form ke `/laporan/create` dan memberi nama route yang berbeda (`laporan.create` dan `laporan.index`). Ketiga, saya memastikan setiap directive ditutup dengan benar (`@if` dengan `@endif`, `@forelse` dengan `@endforelse`) agar tidak terjadi galat sintaks saat Blade dikompilasi. Keempat, karena data pada method `store()` langsung dikirim ke view tanpa disimpan, halaman konfirmasi akan kehilangan data jika di-refresh; hal ini dicatat sebagai keterbatasan prototipe yang dapat diperbaiki dengan menyimpan data sementara menggunakan `redirect()->with('laporan', ...)`.

Kesesuaian dengan ketentuan tugas. Aplikasi LaporBanjir telah memenuhi ketentuan pada bagian 3.5: layout utama `layouts/app.blade.php` berisi header, menu bernama, footer, dan CSS (poin a); daftar laporan memakai `@forelse` dan status ditentukan dengan `@if/@elseif/@else` (poin b); halaman konfirmasi memakai komponen `<x-alert>` (poin c); kartu laporan ditampilkan melalui partial dengan `@include` (poin d); dan setiap halaman dapat diakses melalui route (poin e).

6. Kesimpulan

Praktikum Modul 3 berhasil dilaksanakan. Saya dapat menyimpulkan bahwa Blade Template sangat membantu dalam menyusun tampilan Laravel secara terstruktur. Fitur layouting (`@extends`, `@section`, `@yield`) membuat bagian tampilan yang sama cukup ditulis sekali sehingga setiap halaman terlihat seragam dan mudah dikembangkan. Directive percabangan (`@if/@elseif/@else`) dan perulangan (`@foreach/@forelse`) memudahkan penyajian data dinamis yang dikirim dari controller, seperti status genangan pada aplikasi LaporBanjir. Penggunaan komponen (`<x-alert>`) dan partial (`@include`) juga terbukti membuat kode tidak ditulis berulang-ulang serta lebih mudah dirawat. Pada akhirnya, seluruh tampilan aplikasi LaporBanjir berhasil dibangun dengan Blade Template dan dapat dijalankan dengan baik di browser.